

**Министерство науки и высшего образования Российской
Федерации** ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «**Национальный исследовательский
университет ИТМО**» (Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые
технологии **Направление подготовки** 09.03.03
Прикладная информатика

О Т Ч Е Т
**Отчёт по курсовой работе дисциплины Мобильная
разработка (Android и IOs)**

Обучающийся:
Ананьев Никита Владимирович, К3440

Преподаватель:
Шатинский Григорий Сергеевич

Санкт-Петербург
2026

АННОТАЦИЯ

Темой данной курсовой работы является разработка учебного Android-приложения с помощью ЯП Kotlin, реализующего базовый функционал мессенджера. Приложение позволяет пользователю просматривать список сообщений, новостей, а также редактировать данные профиля и базовые настройки. Разработка велась последовательно в виде четырёх лабораторных работ. В первой работе был создан интерфейс приложения и реализована навигация между экранами. Далее в проект была внедрена архитектура MVVM, а управление состоянием экранов организовано с использованием ViewModel и LiveData. В 3-й и 4-й лабораторной была реализована загрузка данных из публичного REST-API и сохранение сообщений в локальной базе данных Room, что обеспечило возможность работы приложения в офлайн-режиме. Также в последней работе был добавлен список сообщений с возможностью ставить лайки, реализована фоновая периодическая загрузка сообщений из публичного API с помощью WorkManager и отправка уведомлений.

Для выполнения курсовой работы были применены следующие инструменты: Kotlin, среда разработки Android Studio, а также компоненты Android Jetpack (ViewModel, LiveData, Navigation, Room, WorkManager) и библиотека Retrofit для работы с публичными API. Пользовательский интерфейс разработан на основе принципов Material Design. Программа наглядно реализует полный цикл обработки данных: загрузку информации из интернета, её сохранение в локальной базе данных, отображение пользователю, а также фоновое обновление контента с отправкой уведомлений.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
ОСНОВНАЯ ЧАСТЬ.....	5
1 Цели и задачи работы.....	5
2 Описание проекта.....	6
2.1 Общее описание приложения.....	6
2.2 Используемые технологии.....	6
2.3 Архитектура приложения.....	7
3 Выполнение лабораторных работ.....	9
3.1 Лабораторная работа №1. Интерфейс и навигация.....	9
3.2 Лабораторная работа №2. Управление состоянием через ViewModel..	10
3.3 Лабораторная работа №3. Внешний API и база данных.....	11
3.4 Лабораторная работа №4. Фоновые задачи и уведомления.....	15
ЗАКЛЮЧЕНИЕ.....	20

ВВЕДЕНИЕ

На сегодняшний день мобильные приложения являются неотъемлемой частью повседневной жизни многих пользователей, а мессенджеры и социальные платформы занимают одно из ключевых мест среди наиболее востребованных приложений. Развитие Android-платформы и появление современных инструментов разработки позволяют создавать масштабируемые и удобные приложения с использованием актуальных архитектурных подходов и технологий. В связи с этим изучение процесса разработки Android-приложений с применением современных средств является актуальной задачей для студентов, обучающихся по направлениям, связанным с программированием и информационными технологиями.

Целью данной курсовой работы является разработка учебного Android-приложения, реализующего базовый функционал мессенджера и новостной платформы. В рамках работы создается приложение, позволяющее пользователю просматривать новостную ленту с возможностью поиска, обмениваться сообщениями с другими пользователями, управлять данными профиля, а также настраивать параметры приложения. Основное внимание уделяется организации корректной работы с данными, построению архитектуры приложения и использованию современных подходов разработки под Android.

В ходе выполнения курсовой работы были применены современные средства работы с данными и асинхронностью, такие как корутины, Flow и LiveData, что повышает отзывчивость интерфейса и упрощает управление состоянием приложения. Были рассмотрены основные этапы разработки Android-приложения: проектирование пользовательского интерфейса, реализация навигации между экранами, организация взаимодействия с сетью и локальным хранилищем данных, а также обеспечение фонового обновления информации. Полученные результаты позволяют на практике закрепить теоретические знания и продемонстрировать полный цикл разработки мобильного приложения на платформе Android.

ОСНОВНАЯ ЧАСТЬ

1 Цели и задачи работы

Целью данной курсовой работы является разработка учебного Android-приложения, демонстрирующего базовый функционал мессенджера и новостной ленты, с использованием современных архитектурных подходов и инструментов платформы Android, а также освоение полного цикла работы с данными: от получения информации из сети до ее локального хранения и фонового обновления.

Для достижения поставленной цели в ходе выполнения курсовой работы необходимо решить следующие задачи:

- Разработать базу Android-приложения, включающую в себя главную Activity, разметку экранов и навигацию между ними с использованием `nav_graph`.
- Реализовать архитектуру MVVM для фрагментов, обеспечив хранение и обновление пользовательских данных с помощью `ViewModel` и `LiveData`.
- Организовать сетевое взаимодействие с удалённым API для загрузки данных, а также реализовать их сохранение в локальной базе данных `Room`.
- Внедрить слой репозитория и объекты доступа к данным (DAO) для разделения логики работы с сетью и базой данных.
- Реализовать механизм фонового выполнения задач с использованием `WorkManager` для периодического обновления данных.
- Добавить систему уведомлений для информирования пользователя о получении новых данных или обновлениях в приложении.

2 Описание проекта

2.1 Общее описание приложения

Разрабатываемое приложение представляет собой учебный проект в виде мессенджера. В нем пользователь может просматривать ленту сообщений, имитирующих диалоги, а также новостную ленту. Отдельно реализованы экран профиля для изменения личных данных, а также экран настроек с выбором темы.

Все данные загружаются из открытых API-сервисов, предоставляющего тестовые / реальные данные в формате JSON. Сообщения загружаются с <https://dummyjson.com/posts>, а новости с <https://newsapi.org/>. Такой подход позволяет продемонстрировать сетевые запросы без необходимости разворачивать собственный бэкенд-сервер. Для обеспечения работы в условиях отсутствия сети, сообщения сохраняются в локальной базе данных.

2.2 Используемые технологии

В процессе разработки учебного Android-приложения были использованы следующие технологии и инструменты:

- Kotlin – современный язык программирования, рекомендованный для разработки приложений под платформу Android, обеспечивающий простой и безопасный код.
- Android Studio – официальная среда разработки для создания, отладки и тестирования Android-приложений.
- Android Jetpack – набор библиотек и компонент, упрощающих разработку и поддерживающих современные архитектурные подходы:
- Retrofit — библиотека для использования HTTP-запросов и взаимодействия с внешними API.
- Gson – библиотека для сериализации и десериализации данных в формате JSON при работе с сетевыми запросами.
- Kotlin Coroutines – средство для асинхронного выполнения операций без блокировки потока.

- Flow – инструмент для работы с потоками данных и реактивного обновления состояния приложения.
- Material Design – набор рекомендаций и компонент для построения современного и удобного пользовательского интерфейса.

2.3 Архитектура приложения

В проекте используется архитектурный подход MVVM (Model–View–ViewModel), который обеспечивает разделение ответственности между пользовательским интерфейсом, логикой представления и слоем данных.

Точкой входа в приложение является класс `MainApp`, который выполняет начальную конфигурацию приложения и запуск фонового обработчика `MessageWorker`. Он используется для выполнения фоновых запросов к сети и отправки уведомлений пользователю.

За пользовательский интерфейс отвечают четыре основных фрагмента: новости, сообщения, профиль и настройки. Навигация между фрагментами осуществляется с помощью `nav_graph`.

Фрагменты новостей и сообщений через `viewModel` взаимодействуют с репозиториями (`NewsRepository` и `MessagesRepository`), которые инкапсулируют логику получения данных из сети и бд. Для локального хранения данных применяются объекты доступа к данным `NewsDay` и `Messages Do`, реализованные с помощью библиотеки `Room`. Такой подход позволяет разделить источники данных и упростить масштабирование приложения. На рисунке 1 показана схема архитектуры приложения.

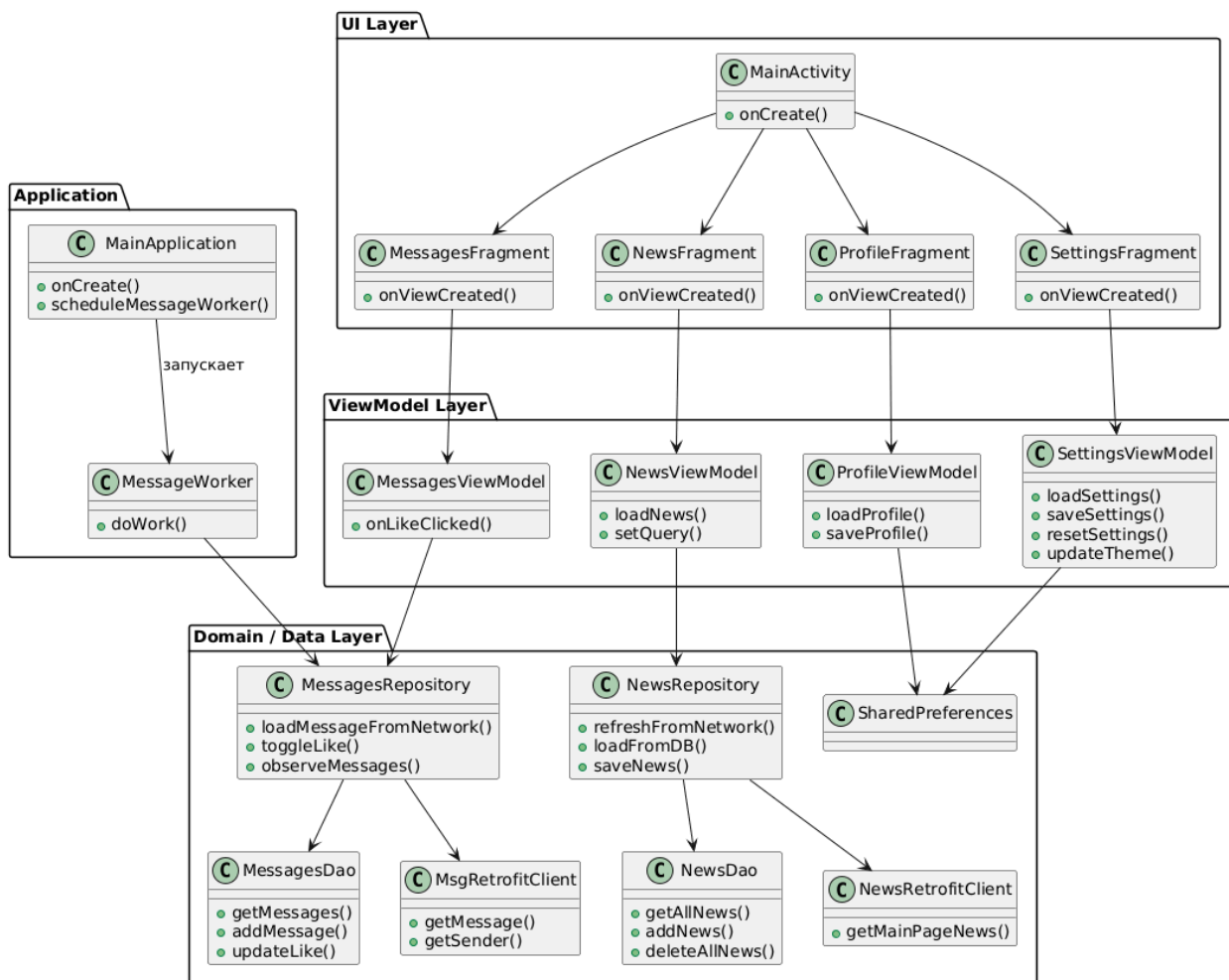


Рисунок 1 – Архитектура приложения

3 Выполнение лабораторных работ

3.1 Лабораторная работа №1. Интерфейс и навигация

В рамках первой лабораторной работы была реализована базовая архитектура приложения. Основой стала главная активность MainActivity, содержащая NavHostFragment – контейнер для управления переключением фрагментов.

Навигационная панель внизу экрана создана с помощью BottomNavigationView. Её пункты: “Новости”, “Сообщения” “Профиль” и “Настройки” описаны в XML-ресурсе menu_bottom_menu.xml.

Связь панели навигации с контроллером переходов настроена в MainActivity через метод setupWithNavController, что позволяет осуществлять переключение между экранами при выборе вкладки без создания дополнительной логики обработки нажатий.

На рисунке 2 показан внешний вид приложения после выполнения 1-й работы.

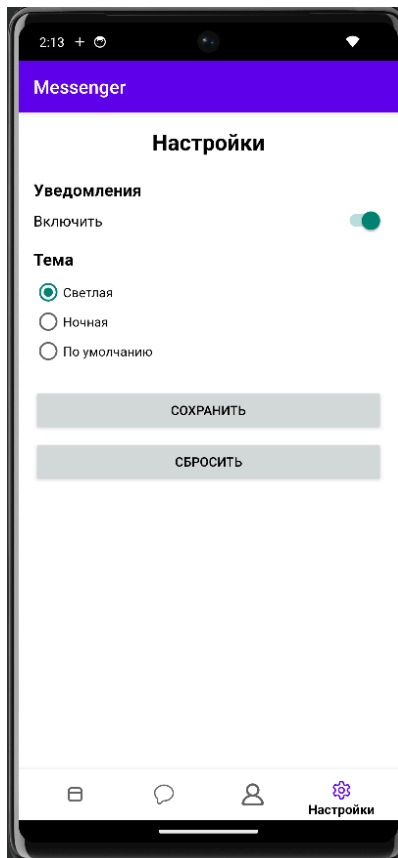


Рисунок 2 – Внешний вид приложения после ЛР1

3.2 Лабораторная работа №2. Управление состоянием через ViewModel

В ходе второй лабораторной работы в архитектуру приложения был добавлен компонент ViewModel из паттерна MVVM и инкапсулировал работу с sharedPreferences внутри фрагментов (использовались для сохранения данных юзера и настроек).

Были созданы ProfileViewModel и SettingsViewModel. В них с помощью MutableLiveData хранятся данные пользователя и настройки. LiveData объект же используется только для реактивного отображения в фрагментах и защищен от изменения. Во фрагментах установлены наблюдатели за LiveData, которые обновляют содержимое полей EditText и RadioButton только в том случае, если их значение действительно изменилось.

Таким образом, логика хранения и изменения данных была вынесена из фрагментов во ViewModel. При повороте устройства состояние профиля и настроек сохраняется, согласно требованиям к работе.

На рисунке 3 изображен код из ProfileViewModel, который отвечает за сохранение и загрузку данных пользователя.

```

private fun saveProfile(profile: UserData) {
    try {
        val json = gson.toJson(profile)
        prefs.edit().putString("user_profile", json).apply()
        _profile.value = profile
        _toastMessage.value = "Profile saved"
    } catch (e: Exception) {
        _toastMessage.value = "Error saving profile"
    }
}

}

nikita-itmo-gh-acc
private fun loadProfile() {
    try {
        val json = prefs.getString(KEY_PROFILE, defValue: null)
        Log.d(tag, msg: "Загрузка профиля: $json")

        if (!json.isNullOrEmpty()) {
            val loadedProfile = gson.fromJson(json, UserData::class.java)
            _profile.value = loadedProfile
        } else {
            _profile.value = UserData()
            Log.d(tag, msg: "Профиль не найден, создан пустой")
        }
    } catch (e: Exception) {
        Log.e(tag, msg: "Ошибка загрузки профиля: ${e.message}")
        _profile.value = UserData()
        _toastMessage.value = "Error loading profile"
    }
}

}

```

Рисунок 3 – Фрагмент из ProfileViewModel

3.3 Лабораторная работа №3. Внешний API и база данных

В ходе выполнения 3-й лабораторной работы была реализована логика взаимодействия с внешним API и локальным хранилищем Room. Приложение получает список сообщений с внешнего сервиса и сохраняет их в базе данных. В качестве удалённого API использовался публичный сервис <https://newsapi.org/>.

Для описания сетевых запросов был создан интерфейс NewsApi, реализованный с помощью библиотеки Retrofit. Главным методом данного

интерфейса является `getMainPageNews`, возвращающий список объектов новостей `NewsData` (см. рисунок 4).

```
8
  nikita-itmo-gh-acc
9 interface NewsApi {
  nikita-itmo-gh-acc
10   @GET("v2/everything?apiKey=$API_KEY&pageSize=25")
11   suspend fun getMainPageNews(
12       @Query("q") query: String,
13       @Query("language") language: String
14   ): NewsResponse
15 }
16
```

Рисунок 4 – Интерфейс `NewsApi`

Модель `NewsData` содержит основные данные публикации, включая уникальный идентификатор публикации, имя автора, дату публикации, заголовок и текст публикации, ссылку на связанное изображение.

Для хранения информации на устройстве пользователя была разработана сущность `NewsEntity`, помеченная аннотацией `@Entity`. Структура таблицы практически точь в точь повторяет данные, получаемые с сервера (см. рисунок 5).

```

7  @Entity(tableName = "news")
8  data class NewsEntity (
9      @PrimaryKey(autoGenerate = true)
10     val id: Int = 0,
11     val author: String?,
12     val title: String,
13     val date: String,
14     val description: String,
15     val imageURL: String,
16     val contentURL: String
17 )
18

```

Рисунок 5 – сущность NewsEntity

Для доступа к данным базы был определен интерфейс NewsDao, в котором описаны основные операции работы с таблицей: получение списка всех новостей, вставка набора записей с заменой существующих данных и обновление отдельной записи. Также был создан класс MessengerDb наследуется от RoomDatabase и предоставляет доступ к DAO через абстрактное свойство.

Логика выбора источника данных и управления ими сосредоточена в классе NewsRepository. Репозиторий принимает на вход сетевой API и объект доступа к базе данных. При выполнении принудительного обновления данных репозиторий инициирует запрос к серверу, преобразует полученные data-объекты в сущности базы данных и сохраняет их в Room. После этого данные считываются из локального хранилища и передаются в вызывающий компонент. В случае возникновения ошибки сетевого соединения репозиторий возвращает данные, сохраненные в локальной базе, что обеспечивает стабильную работу приложения.

Внешний вид фрагмента новостей показан на рисунке 6.

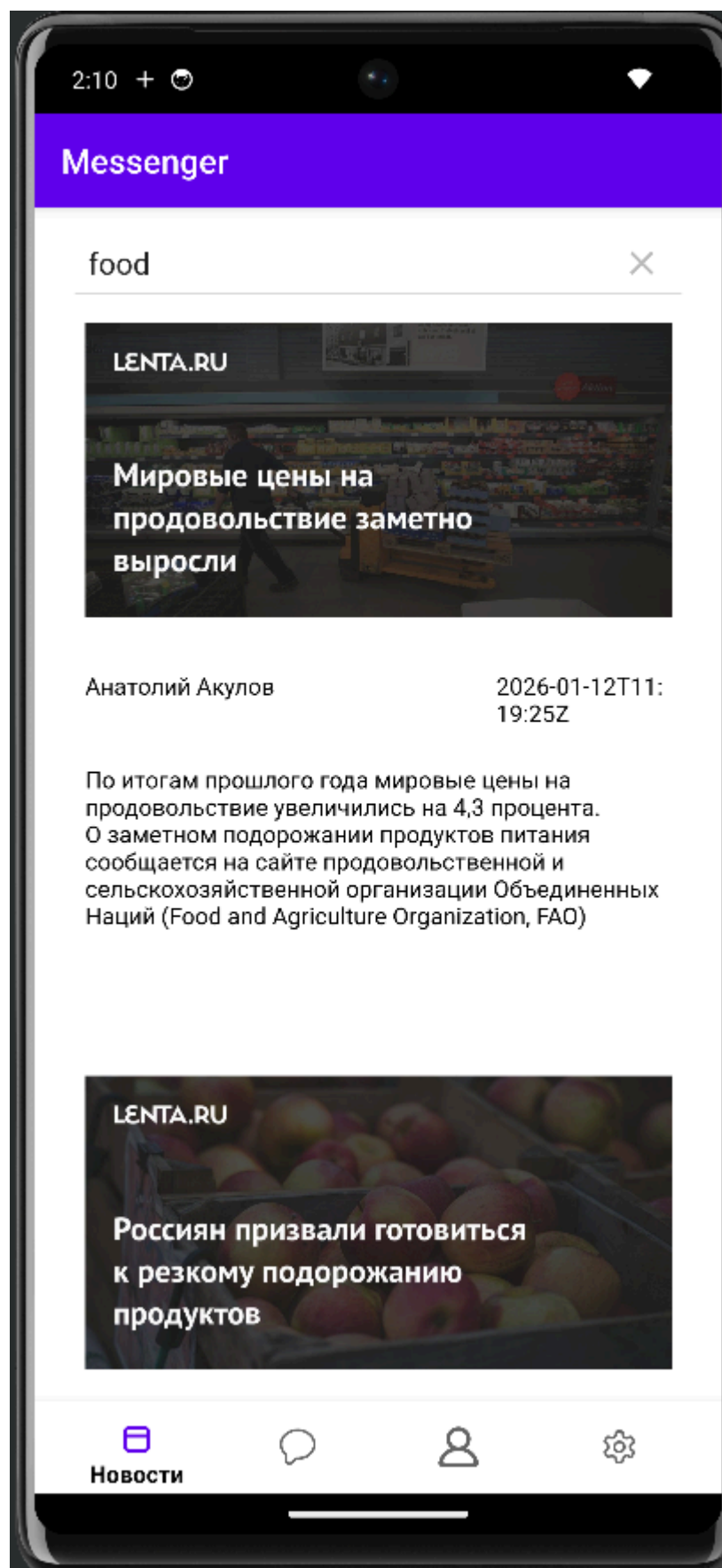


Рисунок 6 – Вкладка с новостями

3.4 Лабораторная работа №4. Фоновые задачи и уведомления

Целью 4-й (последней) лабораторной работы стало улучшение пользовательского опыта за счет внедрения фонового выполнения задач и отправки уведомлений.

Для фрагмента сообщений были реализованы все те же сущности и компоненты что и для новостей, то есть: RecyclerView адаптер, ViewModel, Retrofit клиент, репозиторий и DAO. Модель данных для сообщения показана на рисунке 7.

```
5 data class MessageResponseData (  
6     val id: Int,  
7  
8     @SerializedName("userId")  
9     val senderId: Int,  
10  
11     @SerializedName("body")  
12     val text: String?,  
13 )  
14  
15 data class MessageData (  
16     val id: Int,  
17     val text: String?,  
18     val username: String?,  
19     val avatarURL: String,  
20     val liked: Boolean = false  
21 )  
22
```

Рисунок 7 – Модель данных для сообщения

Важно отметить, что MessageResponseData используется только для получения сетевых данных, внутренняя логика приложения работает только с MessageData.

Для обработки нажатия кнопки лайка в MessageAdapter передается функция onLikeClick. Эта функция является методом ViewModel и позволяет сохранить данные в бд. На рисунке 8 показана логика обработки лайка в MessagesViewHolder.

```
class MessagesAdapter(private val onLikeClick: (MessageData) -> Unit)
    : RecyclerView.Adapter<MessagesAdapter.MessagesViewHolder>() {
    class MessagesViewHolder(val binding: MessageItemBinding): RecyclerView.ViewHolder() {
        fun bind(
            message: MessageData,
            onLikeClick: (MessageData) -> Unit
        ) = with(binding) { this: MessageItemBinding

            userName.text = message.username
            messageText.text = message.text

            likeButton.isSelected = message.liked

            likeButton.setOnClickListener { it: View!
                onLikeClick(message)
            }

            Glide.with(root).load(message.avatarURL)
                .error(R.drawable.default_pfp)
                .placeholder(R.drawable.default_pfp).into(avatarImage)
        }
    }
}
```

Рисунок 8 – Обработка лайка в MessagesViewHolder

Для имитации отправки сообщения от других пользователей был создан MessageWorker. MessageWorker наследуется от CoroutineWorker(context, params) и используется для выполнения фоновых задач. В частности, в лабораторной работе он используется для выполнения периодических запросов на получение сообщения из внешнего API. Сообщение сохраняется в бд, и благодаря Flow потоку сообщений, объявленных в MessageDao, сразу появляется в UI. Помимо создания

сообщения, пользователю отправляется сообщение через объект класса Notifier. Код MessageWorker и Notifier показан на рисунках 9 и 10.

```
13 class MessageWorker(  
14     context: Context,  
15     params: WorkerParameters  
16 ) : CoroutineWorker(context, params) {  
17     @nikita-itmo-gh-acc  
18     companion object {  
19         const val tag = "MessageWorker"  
20     }  
21  
22     @nikita-itmo-gh-acc  
23     override suspend fun doWork(): Result {  
24         return try {  
25             Log.d(tag, msg: "Do work...")  
26  
27             val repository = createRepository()  
28             val notifier = createNotifier()  
29  
30             repository.loadMessageFromNetwork()  
31             notifier.sendNotification()  
32  
33             Result.success()  
34         } catch (e: Exception) {  
35             Log.e(tag, msg: "Worker error: ${e.message}")  
36             Result.retry()  
37         }  
38     }  
39  
40     new *  
41     private fun createNotifier(): Notifier {  
42         return Notifier(applicationContext).apply { this: Notifier  
43             setupNotificationChannel()  
44         }  
45     }  
46 }
```

Рисунок 9 – MessageWorker

```

class Notifier(private val appContext: Context) {
    companion object {
        const val channelId = "notify_channel"
        const val notificationId = 1001
        const val tag = "Notifications"
    }

    fun setupNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val name = "Сообщения"
            val descriptionText = "Уведомления о новых сообщениях"
            val importance = NotificationManager.IMPORTANCE_DEFAULT
            val channel = NotificationChannel(channelId, name, importance).apply {
                description = descriptionText
            }

            val notificationManager = appContext
                .getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager

            notificationManager.createNotificationChannel(channel)
        }
    }

    fun sendNotification() {
        val notification = NotificationCompat.Builder(appContext, channelId)
            .setContentTitle("Новые сообщения")
            .setContentText("У вас есть новые непрочитанные сообщения")
            .setSmallIcon(R.drawable.plus_msg)
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true)
            .build()

        val notificationManager = appContext
            .getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager

        notificationManager.notify(notificationId, notification)
    }
}

```

Рисунок 10 – Notifier

При первом входе пользователя в приложение, у него будет запрошено разрешение на отправку уведомлений. После того как пользователь подтверждает разрешение, фоновый воркер может показывать уведомления о том, что новые сообщения успешно получены.

На рисунке 11 показан внешний вид фрагмента с сообщениями.

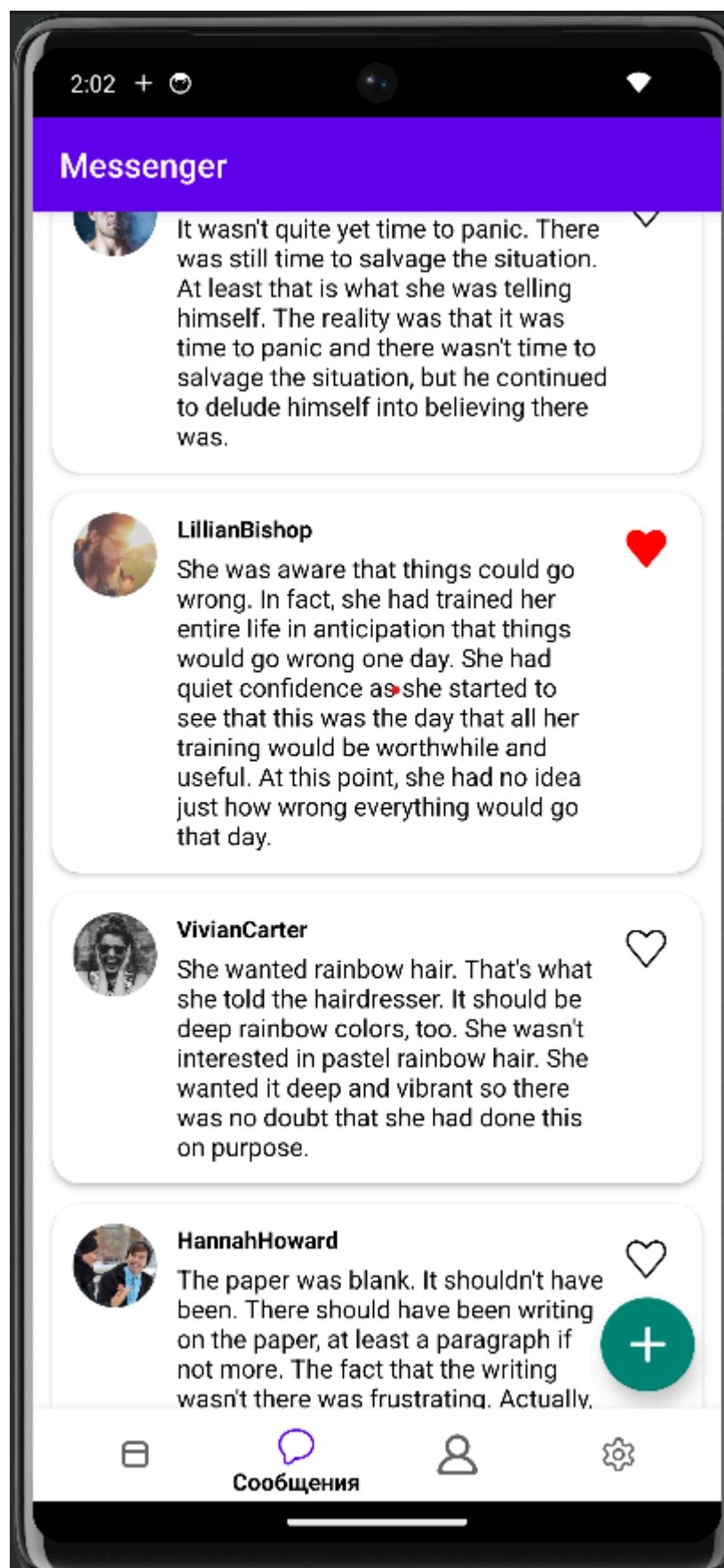


Рисунок 11 – Вкладка с сообщениями

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было реализовано учебное Android-приложение с базовым функционал мессенджера и новостной ленты. В процессе удалось пройти разные этапы создания мобильного приложения, от проектирования UI до организации сетевого взаимодействия, локального хранения данных и фоновое обновления информации.

Ко всему прочему выполнение лабораторных позволило получить и закрепить знания по разработке приложений под Android с использованием языка программирования Kotlin. Была освоена архитектура MVVM, которая позволила разделить пользовательский интерфейс и слой хранения данных.

Особое внимание в работе уделено организации работы с данными. Было реализовано получение данных из внешнего API, с последующей обработкой и сохранением в локальной базе данных Room.

Кроме того, в ходе работы рассматривались механизмы фоновое выполнения задач с использованием WorkManager и была создана система уведомлений для информирования пользователя о новых сообщениях.

В конечном счете удалось реализовать проект с продуманной архитектурой, пригодной для масштабирования, и демонстрирующий полный цикл работы с данными в Android-приложении.